

AI Skincare Progress Coach

Complete build and launch plan for a solo, AI-assisted developer

Stack: React Native (Expo) + Supabase + Gemini vision

Ship order: Google Play first, iOS second

Build environment: Windows / Linux, no Mac required

Contains: 34 copy-paste prompts for Claude Code, full database schema, the vision system prompt, store compliance rules, pricing and launch playbook

Version 1.0

How to use this document

This is a build manual, not a summary. It is written to be worked through in order, from an empty folder to a live listing on Google Play. Every phase contains numbered prompts you paste directly into Claude Code.

The three things that will kill this project

Before anything else, internalise these. Everything in this document is arranged around them.

- 1. The Google Play 14-day gate.** A new personal Play developer account cannot publish to production until 12 testers have been opted into a closed test **continuously for 14 days**. That clock cannot be shortened or bought out. If you wait until the app is finished to start it, you add three to five weeks to your launch. Phase 14 tells you to start that clock in **week 3**, on a deliberately unfinished build.
- 2. Medical language.** The moment your app copy says "diagnose", "condition", "treat" or "dermatologist-grade accuracy", you are a medical app in the eyes of both stores, and you need regulatory clearance you do not have. The banned-vocabulary list in Appendix A goes into your **CLAUDE.md** so that every string Claude Code writes is compliant by default.
- 3. Retention, not installs.** AI apps churn roughly 30% faster than non-AI apps. A skin *scanner* is a novelty someone uses once. A skin *progress tracker* is a subscription. Every design decision in this plan protects the re-scan loop. Do not let feature creep take engineering time away from it.

Document conventions

Marker	Meaning
P4.2	Prompt 4.2 — paste the boxed text into Claude Code verbatim
Done when	Acceptance criteria. Do not move on until these are all true
Commit	The git commit message to use, so your history stays readable
Teal callout	Context, reasoning, or a tip
Red callout	A trap that costs real time or money if ignored

How to work with Claude Code effectively

These habits matter more than the prompts themselves.

- 1. One prompt, one concern.** The prompts below are deliberately scoped. Resist merging three of them into one message; long multi-goal prompts produce sprawling diffs you cannot review.
- 2. Commit after every prompt that passes its Done-when.** This is your undo button. If a later prompt makes a mess, **git reset --hard** costs you one prompt instead of a day.
- 3. Work on a branch per phase.** **git checkout -b phase-06-scan-pipeline**. Merge to **main** when the phase is done and the app still runs.
- 4. Always read the diff before accepting.** You do not need to understand every line, but you must notice when it rewrites a file you did not ask it to touch.
- 5. When something breaks, paste the entire error.** Terminal output, stack trace, and what you did just before. Truncated errors produce guessed fixes.
- 6. Say "don't do that" early.** If Claude Code starts installing a state library you did not ask for, stop it in the same message rather than letting it finish.

7. **Keep CLAUDE.md current.** It is read at the start of every session. When a convention changes, update the file rather than repeating yourself in prompts.
8. **Never let secrets into the repo.** If you ever see an API key in a diff, stop, remove it, and rotate the key. Assume anything committed to GitHub is public forever.

On the working name. This document uses **GlowTrack** as a placeholder and **com.yourname.glowtrack** as the bundle identifier. Pick your real name and bundle ID *before Phase 1*. The bundle ID is permanent — it cannot be changed after your first Play Console upload without creating an entirely new listing. Check name availability on both stores and as a **.com** domain before committing.

Your context block

Fill this in before you start. Several prompts later in the document ask you to paste it in, so that Claude Code has the project facts it needs without you retyping them.

PROJECT CONTEXT

=====

App name: _____

Bundle / package ID: _____
(e.g. com._____._____)

Domain (for privacy policy + landing page): _____

GitHub repo URL: _____

Supabase project ref: _____

Primary target markets: _____
(which countries you will price for)

Launch target date: _____

Closed-test start date: _____
(launch date minus 21 days, minimum)

DEVELOPER PROFILE

=====

Coding experience: _____

Hours available per week: _____

Android test device (make / model / OS): _____

Access to an iPhone for later iOS testing? _____

ACCOUNTS CREATED

=====

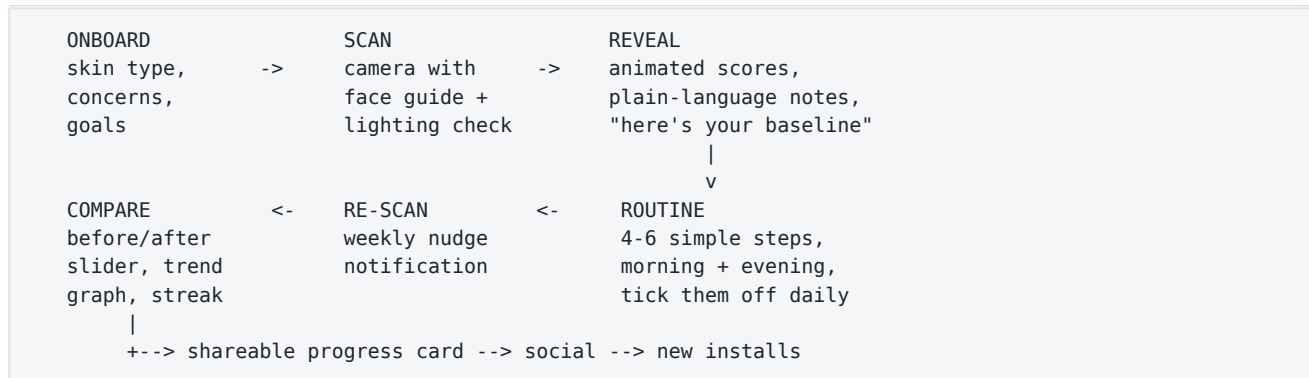
☐ GitHub	☐ Supabase
☐ Google AI Studio	☐ Expo (EAS)
☐ RevenueCat	☐ Google Play Console
☐ PostHog	☐ Sentry

1. What you are building

The product in one paragraph

A mobile app that photographs your face, returns a set of cosmetic skin scores within seconds, gives you a simple daily routine, and then — the part that actually matters — tracks those scores over weeks so you can finally see whether your routine is working. The scan is the hook. The progress graph is the product.

The core loop



The loop closes on itself. That is the whole thesis: a one-shot analyser has no second act, while "is it working yet?" is a question a user wants answered every single week.

The wedge

The category splits into two groups, and both leave a gap.

Incumbent type	Examples	Weakness you exploit
Looksmaxxing / face-rating	Umax, LooksMax, Youmax	Creepy framing, aimed at insecure teenage boys, brutal churn once you have seen your score
Skin analysers	Skan, Dr. Skin AI, Nolla	One-shot analysis, product-pushing, no real longitudinal tracking
Brand tools	L'Oreal, Charlotte Tilbury	Locked to one brand's catalogue, not a neutral tracker

Your position: **honest, gender-neutral, progress-first**. Not "how attractive are you" but "is your routine working". That framing widens the market, lowers platform and PR risk, and is the thing none of the incumbents own.

Scope discipline: what v1 is NOT

Write this list on a sticky note. Every one of these is a plausible, tempting feature that will cost you the launch window.

- Not a product recommender with affiliate links (v1.2 at the earliest)
- Not an ingredient scanner or barcode reader
- Not a dermatologist marketplace or chat-with-a-doctor feature
- Not a social feed, friends list, or leaderboard
- Not an AI chatbot you can ask skincare questions
- Not multi-language (English only for v1)

- Not a web app
- Not iOS (not yet — see Phase 15)

Feature set by release

Release	Features
MVP (launch)	Onboarding quiz, camera capture with lighting and face guides, AI scan returning 7 cosmetic scores, results screen with animated reveal, scan history timeline, before/after comparison slider, generated routine with daily check-off, streaks, weekly re-scan reminder, hard paywall after first free scan, account deletion
v1.1 (weeks 2-6 post-launch)	Shareable progress card, routine editing, multiple photo angles, trend graph per attribute, referral loop, widget
v1.2+	Product logging ("I started using X on this date" overlaid on the trend graph), affiliate recommendations, iOS release, additional languages

The single most valuable v1.1 feature is the shareable progress card. It converts your existing users into your marketing channel. It is excluded from MVP only because the 14-day Play gate means you should be shipping, not polishing. Build it while the closed test runs.

2. Locked technical decisions

These are settled. The prompts in this document assume them. Changing one means rewriting the phases that depend on it.

Layer	Choice	Why
Framework	React Native via Expo , TypeScript	One codebase for Android and iOS. Expo removes almost all native build configuration, works fully on Windows, and builds iOS in the cloud later without a Mac
Routing	Expo Router (file-based)	Fewer moving parts than React Navigation configured by hand; Claude Code produces more consistent output against it
Styling	StyleSheet + a central theme token file	Zero build configuration and zero risk of a broken Tailwind pipeline. Deliberately not NativeWind for v1
State	Zustand	~1 KB, no boilerplate, no providers to wire
Backend	Supabase	Postgres + Auth + Storage + Edge Functions in one product, generous free tier, row-level security you can actually read
Vision model	Gemini Flash-Lite tier (see below)	Cheapest credible vision model with native structured-JSON output
Subscriptions	RevenueCat	Free below \$2,500 monthly tracked revenue, then 1%. Removes receipt validation and entitlement sync entirely
Analytics	PostHog	Generous free tier, funnels and cohorts without a data team
Crash reporting	Sentry	Free tier is sufficient at your volume
Builds	EAS Build (Expo's cloud)	Builds Android and iOS from Windows. Free tier gives 15 Android + 15 iOS builds/month
CI	GitHub Actions	Typecheck and lint on every push; keeps Claude Code honest

Corrections to the research document

The research that led to this build is sound on strategy, but three specifics have moved. Do not follow the research doc on these points.

Gemini 2.5 Flash is being retired on 16 October 2026. The research document recommends it. Do not build on it. Use the current Flash-Lite tier instead — as of September 2026 that is Gemini 3.1 Flash-Lite (roughly \$0.25 per million input tokens / \$1.50 output), with Gemini 3.5 Flash-Lite as the step up if quality is short. Your abstraction layer (Phase 6) exists precisely so this swap is a one-line change.

Ship Android first, not iOS first. The research recommends iOS-first for higher willingness to pay, and that is correct in the abstract. It is wrong for you: you have no Mac and no iPhone, so you cannot test an iOS build even though EAS can compile one. Ship to Play, validate the loop with real users and real money, then port. Phase 15 covers the iOS path.

App category is now a compliance decision. Google explicitly lists "skin care" under its **Medical** category definition, which triggers a mandatory Health apps declaration in Play Console. List under **Beauty**, keep every string non-medical, and you stay out of that regime. Get this wrong and you inherit a documentation burden designed for regulated medical devices.

Cost model at three scales

All figures in ZAR, converted at roughly R18 to the dollar. Store commission is the dominant cost; inference is a rounding error.

Line item	Pre-launch	1,000 subscribers	10,000 subscribers
Supabase	R0 (free tier)	R450/mo (Pro)	R450/mo + usage
Gemini vision API	R0-R50	~R700/mo	~R7,000/mo
Expo EAS	R0 (free tier)	R0-R350/mo	R350/mo
RevenueCat	R0	R0 (under threshold)	~1% of gross
PostHog / Sentry	R0	R0	R0-R500/mo
Play Console	R450 once	—	—
Apple Developer (later)	R1,800/yr	—	—
Google's 15-30% cut	—	dominant line	dominant line

Inference will not hurt you. At roughly 1,900 input tokens (a 1024px image is about 1,290 tokens, plus your system prompt) and 500 output tokens per scan, a single scan costs in the region of R0.02. Even at 30 scans per user per month that is under R1 per user per month against a subscription of R80-R90. Your real costs are the store commission and customer acquisition. Do not optimise the model spend; optimise conversion and retention.

3. Critical path and timeline

This assumes roughly 12-18 hours a week. Halve your available hours and add 50% to the calendar, not 100% — Claude Code compresses the build but not the waiting.

Week	Build work	Store / business work
0	Phase 0: environment, accounts, repo	Decide name + bundle ID. Register Play Console account immediately (the 14-day clock cannot start until it exists)
1	Phases 1-3: scaffold, design system, Supabase schema	Reserve the app name, buy the domain
2	Phases 4-6: auth, onboarding, camera, scan pipeline	Draft privacy policy and terms; publish them to a live URL
3	Phase 7: results screen. First EAS build.	Start the closed test. Recruit 12 testers. The clock starts now
4	Phases 8-9: progress tracking, routine, reminders	Closed test running. Gather feedback, ship updates to the same track
5	Phase 10-11: paywall, RevenueCat, analytics	Set up Play subscription products. Shoot launch video content
6	Phases 12-13: compliance, polish, QA	Day 14 passes. Apply for production access. Build the store listing and screenshots
7	Fix whatever QA found. Build v1.1 shareable card	Google reviews production access (allow up to 7 days). Line up creators
8	Buffer	Launch. Creator posts go live

The single most common way this plan fails is treating the closed test as the last step instead of a parallel track. Your week-3 build will be ugly, missing the paywall, and probably crashy. Ship it to closed testing anyway. Testers can keep receiving updates throughout — what matters to Google is that 12 people stayed opted in for 14 continuous days and actually used the thing.

The 12-tester problem, honestly

You need 12 real people, on real Android devices, with real Google accounts, who opt in and stay opted in. Emulators and duplicate accounts are detected and risk permanent account suspension.

Your realistic options:

1. **Recruit people you know.** Friends, family, colleagues, running club, church, study group. This is free and safest. Budget a full week of asking; roughly half of the people who say yes will never actually opt in.
2. **Skincare communities.** Reddit, Discord and Facebook groups in the skincare niche. Offer free lifetime access in exchange for testing. This doubles as your first user research.
3. **Paid testing services.** These exist, Google's own questionnaire acknowledges paid recruitment as legitimate, and they cost in the region of R350-R900. They get you to 12 in hours instead of weeks. The trade-off is that paid testers give you no useful product feedback.
4. **Register an organisation account instead.** Organisation Play accounts are exempt from the requirement entirely. This needs a registered legal entity and a D-U-N-S number, which is free but can take several weeks to issue. Only worth it if you already have a company, or you plan to ship several apps.

Recommended play: recruit 12-15 from your own network for the product feedback, and hold a paid service in reserve if you are still short after a week. Over-recruit: aim for 15, because a tester who opts out resets their own 14-day clock.

4. Accounts and services checklist

Create all of these in Phase 0. Several have delays that will block you later.

Service	Cost	Notes
GitHub	Free	Private repo. Do this first
Google Play Console	~R450 once	Register on day one. Identity verification can take several days, and until it clears you cannot create the closed test
Supabase	Free tier	500 MB database, 1 GB storage, 50k monthly active users. Free projects pause after a week of inactivity — harmless in development
Google AI Studio	Free tier, then pay-as-you-go	Get the Gemini API key here. Free tier is fine for development; move to the paid tier before launch, both for rate limits and because free-tier data may be used for training
Expo / EAS	Free tier	15 Android + 15 iOS builds per month, low-priority queue. Sufficient through launch
RevenueCat	Free under \$2,500 MTR	Then 1% of gross tracked revenue
PostHog	Free tier	Generous event allowance
Sentry	Free tier	Crash reporting
Domain	~R150/yr	Needed for the privacy policy URL, which must be a live web page, not a PDF
Apple Developer	~R1,800/yr	Only when you start Phase 15. Do not pay for this yet

Register the Play Console account before you write a line of code. Google's identity verification for new developer accounts can take days, and for organisation accounts considerably longer. Everything downstream — the closed test, the 14-day clock, the production application — is gated behind it. This is the single highest-leverage thing you can do today.

Your privacy policy must live at a publicly accessible, non-geofenced URL and must not be a PDF. Google is explicit about this. A simple static page on your domain, or a free host, is fine. It must also be linked from inside the app itself, not just from the store listing.

5. Phase 0 — Environment setup

Goal: a Windows machine that can run an Expo app on your physical Android phone, with Claude Code driving a Git repository.

5.1 Install the toolchain

Work through these in order. Each one takes a few minutes.

#	Install	How / notes
1	Node.js LTS	nodejs.org, the LTS build. Verify with <code>node -v</code> and <code>npm -v</code>
2	Git	git-scm.com. Accept the defaults. Verify with <code>git --version</code>
3	VS Code	code.visualstudio.com
4	Claude Code	Follow the current install instructions in Anthropic's docs; run it from the VS Code integrated terminal
5	Expo Go	From the Play Store, onto your physical Android phone
6	Android Studio	<i>Optional.</i> Only needed if you want an emulator. A real phone is better for a camera app — an emulator's fake camera makes the core feature untestable

Configure Git once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Your phone and PC must be on the same Wi-Fi network for Expo Go to connect. If your network blocks device-to-device traffic (common on guest and corporate Wi-Fi), run `npx expo start --tunnel` instead. Tunnel mode is slower but works across networks.

5.2 Verify the setup

Before building anything real, prove the pipeline works end to end.

```
npx create-expo-app@latest scratch-test
cd scratch-test
npx expo start
```

Scan the QR code with Expo Go. If a screen appears on your phone, your environment is correct. Delete the folder afterwards.

If `npx expo start` fails on Windows with a permissions or path error, the usual cause is a project folder inside OneDrive. OneDrive's file syncing breaks `node_modules`. Keep all projects in a plain path such as `C:\dev\`.

5.3 Create the accounts

Work through the table in Section 4. Register the Play Console account now — not later.

For each service, record the credentials somewhere safe. You will need these values later:

SUPABASE_URL	https://xxxx.supabase.co	
SUPABASE_ANON_KEY	eyJ...	(safe in the app)
SUPABASE_SERVICE_ROLE_KEY	eyJ...	(NEVER in the app)
GEMINI_API_KEY	AIza...	(NEVER in the app)
REVENUECAT_ANDROID_KEY	goog_...	(safe in the app)
POSTHOG_API_KEY	phc_...	(safe in the app)
SENTRY_DSN	https://...	(safe in the app)

Learn this distinction now, because getting it wrong is the classic way indie apps get drained. Anything marked "safe in the app" is designed to be public and is protected by other means (row-level security, domain restrictions). Anything marked NEVER must exist only in Supabase Edge Function secrets. A Gemini key shipped inside a mobile app can be extracted from the APK in minutes and billed to you until you notice.

6. Phase 1 — Repository and scaffold

Goal: a running Expo app on your phone, in a private GitHub repo, with a **CLAUDE.md** that makes every later prompt shorter.

P1.1 — Create the repository

Do this manually, before Claude Code touches anything.

```
mkdir C:\dev\glowtrack
cd C:\dev\glowtrack
git init
```

Then create a **private** repo on GitHub named **glowtrack** (no README, no .gitignore — the scaffold provides them), and connect it:

```
git remote add origin https://github.com/YOURNAME/glowtrack.git
```

P1.2 — Scaffold the project

Open Claude Code in that folder and paste this.

PASTE INTO CLAUDE CODE

I'm building a React Native mobile app with Expo. Set up the project foundation in the current directory.

CONTEXT

- Windows dev machine. I test on a physical Android phone via Expo Go.
- No Mac. iOS will come later via EAS cloud builds.
- Solo developer. I can read code but I'm not an experienced React Native dev, so favour clarity over cleverness.
- App name: GlowTrack <- replace with your name
- Package ID: com.yourname.glowtrack <- replace with your bundle ID

TASKS

1. Create an Expo app here using the latest stable Expo SDK, the TypeScript template, and Expo Router for file-based routing.
2. Create this exact folder structure (empty .gitkeep files where needed):

```
app/                                Expo Router routes only
  (auth)/
  (onboarding)/
  (tabs)/
  scan/
src/
  components/ui/
  components/scan/
  components/progress/
  lib/
  stores/
  theme/
  types/
  constants/
supabase/
  migrations/
  functions/
assets/
```

3. Install exactly these and nothing else:
zustand

```
@supabase/supabase-js
expo-secure-store
expo-image
expo-haptics
react-native-svg
Plus whatever Expo Router itself requires.
```

4. Configure TypeScript path aliases so "@/..." maps to "src/...", and make sure both tsconfig and the Metro/Babel config agree.
5. Create .env.example with these keys and empty values:


```
EXPO_PUBLIC_SUPABASE_URL
EXPO_PUBLIC_SUPABASE_ANON_KEY
EXPO_PUBLIC_POSTHOG_KEY
EXPO_PUBLIC_SENTRY_DSN
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY
```
6. Write a .gitignore that excludes: node_modules, .env, .expo, dist, build, *.keystore, *.jks, google-services.json, ios/, android/.
7. Set the app name, slug, package identifier, scheme and a portrait-only orientation lock in app.json.
8. Replace README.md with a short setup guide: prerequisites, how to install, how to run, how to build.

RULES

- Check the actual latest stable Expo SDK version before installing. Do not assume a version number from memory.
- Prefer expo-* packages over third-party equivalents.
- Do NOT install: NativeWind, Tailwind, Redux, React Navigation directly, styled-components, or any UI kit. I want plain StyleSheet.
- Do NOT create any screens or components yet. Structure only.
- When done, run `npx expo-doctor` and fix anything it reports.

Then stop and tell me exactly what command to run and what I should see on my phone.

Done when: `npx expo start` runs, the QR code scans in Expo Go, a blank screen loads on your phone, and `npx tsc --noEmit` passes clean.

Commit: `chore: scaffold expo app with router and folder structure`

P1.3 — Write CLAUDE.md

This is the highest-leverage file in the project. It is read at the start of every Claude Code session, so every rule you put here is a rule you never have to repeat.

PASTE INTO CLAUDE CODE

Create a CLAUDE.md in the project root. This file is read at the start of every session, so it must be complete but tight – no filler.

Include these sections:

1. PROJECT – what the app is in three sentences: a cosmetic skincare progress tracker. The user photographs their face, gets cosmetic appearance scores, and tracks those scores over weeks to see whether their routine is working.
2. STACK – Expo + Expo Router + TypeScript, Zustand for state, Supabase for auth/database/storage/edge functions, RevenueCat for subscriptions, StyleSheet with theme tokens for styling. Android first, iOS later.
3. COMPLIANCE – THE MOST IMPORTANT SECTION. State that this is a BEAUTY app,

not a medical app, and that store approval depends on it. Include:

NEVER use these words in any user-facing string, comment, variable name, commit message, or store copy:

diagnose, diagnosis, condition, disease, disorder, symptom, treat, treatment, cure, heal, therapy, medical, clinical, patient, prescription, dermatologist-grade, medically proven, acne vulgaris, rosacea, eczema, psoriasis, dermatitis, melasma, lesion

ALWAYS use this vocabulary instead:

the appearance of, looks, visible, cosmetic, routine, care, skincare, clarity, texture, evenness, glow

Every screen that shows a score must display the disclaimer from `src/constants/copy.ts`. No exceptions.

4. ARCHITECTURE — the folder structure, what belongs in each folder, and the rule that ALL user-facing strings live in `src/constants/copy.ts` and are imported from there. Never hardcode a user-facing string in a component.
5. CONVENTIONS —
 - Functional components with typed props. No class components.
 - Named exports except for Expo Router route files, which must default-export.
 - No ``any``. Use ``unknown`` and narrow it.
 - Every async call that can fail has explicit error handling and a user-visible error state. No silent catches.
 - Every list has an empty state and a loading state.
 - Colours, spacing, radii and font sizes come from `src/theme/tokens.ts`. Never a hardcoded hex value or magic number in a component.
 - Accessibility labels on every touchable.
6. SECURITY —
 - Only `EXPO_PUBLIC_*` variables may be referenced in app code.
 - The Gemini API key and the Supabase service role key exist ONLY in Supabase Edge Function secrets. If you are ever about to put either in the app, stop and tell me instead.
 - Every table has row-level security enabled.
7. DEFINITION OF DONE — a change is done when: ``npx tsc --noEmit`` passes, the app runs on device without a red screen, loading and error states exist, no new hardcoded strings or colours, and no secrets are in the diff.
8. WHAT NOT TO DO — do not add dependencies without asking; do not refactor files unrelated to the current task; do not add features I did not ask for; do not write tests unless asked; do not change `app.json`, `eas.json` or any migration file without telling me explicitly.

Done when: `CLAUDE.md` exists and you have read it start to finish yourself.

Commit: `docs: add CLAUDE.md project guide`

Re-read `CLAUDE.md` whenever Claude Code starts drifting. Most drift — wrong folder, hardcoded colours, a stray dependency — is a symptom of a rule that is missing or vague in this file. Fix the file, not the individual prompt.

P1.4 — Continuous integration

PASTE INTO CLAUDE CODE

Add a GitHub Actions workflow at `.github/workflows/ci.yml` that runs on every push and pull request to main.

It should:

- Check out the code and set up the Node version from `.nvmrc` (create `.nvmrc` with the LTS major version if it doesn't exist)
- Install dependencies with `npm ci`
- Run ``npx tsc --noEmit``
- Run ESLint

Also set up ESLint and Prettier for an Expo TypeScript project using Expo's own recommended config, and add "lint", "typecheck" and "format" scripts to `package.json`.

Keep the workflow minimal and fast. Don't add tests, build steps, or caching complexity yet.

Done when: you push to GitHub and the Actions tab shows a green check.

Commit: `ci: add typecheck and lint workflow`

7. Phase 2 — Design system

Goal: a coherent visual language and a set of primitives, so every later screen is assembled rather than invented. Doing this now is what stops the app looking like a template.

Art direction. Aim for calm, clinical-but-warm, gender-neutral. Think a quiet wellness product, not a beauty counter and not a gaming app. Avoid pink-and-gold "beauty app" cliché — it halves your market and signals novelty. Deep neutrals, one restrained accent, generous whitespace, one confident typeface. The scores are the only place colour should shout.

P2.1 — Theme tokens

PASTE INTO CLAUDE CODE

Create `src/theme/tokens.ts` as the single source of truth for all visual values.

Design direction: calm, modern, gender-neutral wellness. Not pink, not gold, not clinical-cold. Think quiet confidence — deep neutrals with one restrained accent. The app should feel like a well-made journal, not a beauty counter.

Export:

1. `colors` — a semantic palette (not raw names). Include: `background`, `surface`, `surfaceElevated`, `border`, `borderSubtle`, `textPrimary`, `textSecondary`, `textTertiary`, `textInverse`, `accent`, `accentSubtle`, `accentPressed`, `success`, `warning`, `danger`, and a `'score'` scale of 5 steps from low to high that reads well for someone with colour-vision deficiency (do not rely on red-to-green alone — vary lightness too). Provide both a light and a dark set, and a `'useColors()'` hook that returns the right set for the current system colour scheme.
2. `spacing` — a 4pt scale: `xs 4`, `sm 8`, `md 16`, `lg 24`, `xl 32`, `xxl 48`
3. `radius` — `sm 8`, `md 12`, `lg 20`, `full 999`
4. `typography` — a type scale with `fontSize` and `lineHeight` pairs: `display`, `h1`, `h2`, `h3`, `body`, `bodySmall`, `caption`, `label`. Use the system font for now with a `'fontFamily'` field per style so I can swap in a custom font later by editing one file.
5. `shadows` — three elevation levels that work on both Android (elevation) and iOS (`shadowColor/Offset/Opacity/Radius`).
6. `motion` — standard durations (`fast 150`, `base 250`, `slow 400`) and easing curves for use with Reanimated.

Add a short comment at the top explaining that nothing in the app may use a hardcoded colour, spacing value, radius or font size — everything imports from here.

Done when: `tokens.ts` exists, exports all six groups, and typechecks.

Commit: `feat(theme): add design tokens`

P2.2 — UI primitives

PASTE INTO CLAUDE CODE

Build the core UI primitives in `src/components/ui/`. Every one must use `src/theme/tokens.ts` – no hardcoded values anywhere.

Components:

1. Button – variants: primary, secondary, ghost, destructive.
Sizes: sm, md, lg. Props for loading (shows a spinner, disables input), disabled, fullWidth, and an optional leading icon.
Must have a pressed state with a subtle scale animation and haptic feedback via `expo-haptics`.
2. Card – a surface container with padding, radius and elevation props.
3. Text – a typed wrapper over React Native Text that takes a ``variant`` prop matching the typography scale, plus a ``color`` prop taking semantic colour names. This is how all text in the app gets rendered.
4. Screen – a layout wrapper handling safe-area insets, background colour, optional scroll, and optional keyboard avoidance. Every screen uses this.
5. ScoreRing – an animated circular progress ring built with `react-native-svg`.
Props: score (0-100), size, strokeWidth, label, and animate (boolean).
When animate is true the ring sweeps from 0 to the score over ~900ms and the number counts up in sync. Colour comes from the tokens ``score`` scale based on the value.
6. Chip – a small selectable pill for multi-select, with selected/unselected states.
7. EmptyState – an icon, a headline, a body line, and an optional action button.
8. LoadingState – a simple skeleton shimmer, plus a centred spinner variant.
9. ErrorState – an icon, message, and a retry button.
10. Disclaimer – a small, quiet, always-visible cosmetic-not-medical notice.
It pulls its text from `src/constants/copy.ts`. It must be unmissable but not alarming.

RULES

- Every touchable needs an `accessibilityLabel` and `accessibilityRole`.
- Every component's props interface is exported.
- No component fetches data or knows about Supabase. Presentation only.

Then create `app/dev-gallery.tsx` – a scrollable screen that renders every variant of every component so I can eyeball them all on device. I'll delete this route before release.

Done when: you can open `/dev-gallery` on your phone and see every component, in both light and dark mode.

Commit: `feat(ui): add core component primitives`

P2.3 — The copy file

This file is your compliance chokepoint. Every user-facing word in the app lives here, which means one review of one file proves the whole app is compliant.

PASTE INTO CLAUDE CODE

Create `src/constants/copy.ts` – the single source of every user-facing string in the app.

Structure it as a nested, strongly-typed `const` object grouped by area: onboarding, auth, scan, results, progress, routine, payroll, settings, errors, disclaimers.

CRITICAL COMPLIANCE CONSTRAINT

This is a cosmetic beauty app, not a medical app. Store approval depends on it. No string in this file may contain: diagnose, diagnosis, condition, disease, disorder, symptom, treat, treatment, cure, heal, therapy, medical, clinical, patient, prescription, dermatologist-grade, medically proven, or any named skin condition.

Use instead: "the appearance of", "looks", "visible", "cosmetic", "routine", "care", "clarity", "texture", "evenness", "glow".

TONE

Warm, plain, specific, never hyped, never alarming, gender-neutral. Short sentences. Second person. Assume an intelligent adult who is slightly sceptical of AI claims. Never promise results.

MUST INCLUDE

`disclaimers.short` – one line, shown under every score:
something to the effect that this is a cosmetic estimate from a photo, not medical advice.

`disclaimers.full` – a short paragraph for the results screen and settings, stating plainly that the app gives cosmetic appearance estimates only, that lighting and camera quality affect results, that it is not a substitute for professional advice, and that anything they are worried about should go to a qualified professional.

`disclaimers.referral` – the message shown when a scan is flagged as worth professional attention. It must NOT describe what was seen. It should simply, calmly suggest that a qualified skincare professional is the right person to look at things the app can't assess from a photo.

Add a comment at the top of the file with the banned-word list, so anyone editing it sees the constraint first.

Done when: the file exists and you have personally read every string in it against the banned-word list.

Commit: feat: add centralised copy with compliance constraints

8. Phase 3 — Supabase: database, security and storage

Goal: a schema with row-level security that makes it structurally impossible for one user to read another's face photographs.

P3.1 — Create the Supabase project

Manual steps, done in the Supabase dashboard:

1. Create a new project. Choose the region closest to your main market, not to you — latency for your users matters more than latency for your dev loop.
2. Save the database password somewhere safe.
3. From Settings > API, copy the Project URL and the **anon** key into your **.env**.
4. Copy the **service_role** key somewhere safe and **do not put it anywhere near the app**.

P3.2 — Schema and row-level security

PASTE INTO CLAUDE CODE

Create the database schema as a Supabase migration in
supabase/migrations/0001_initial_schema.sql.

Requirements: Postgres, UUID primary keys with `gen_random_uuid()`, `timestampz` for all timestamps, row-level security enabled on EVERY table, and a policy set where a user can only ever read or write their own rows.

TABLES

profiles

```
id uuid PK, references auth.users(id) on delete cascade
created_at, updated_at
display_name text
age_band text, constrained to: 18_24, 25_34, 35_44, 45_54, 55_plus
skin_type text, constrained to: oily, dry, combination, normal, sensitive, unsure
concerns text[] default '{}'
primary_goal text
timezone text default 'UTC'
onboarding_completed_at timestampz
reminder_enabled boolean default true
reminder_weekday int default 0
reminder_hour int default 19
free_scan_used boolean default false
```

scans

```
id uuid PK
user_id uuid, references auth.users(id) on delete cascade
created_at
image_path text not null -- path within the private storage bucket
status text default 'pending', constrained to:
    pending, processing, complete, failed, rejected
failure_reason text
capture_quality jsonb -- client-side brightness/blur estimates
model text -- which model produced the result
prompt_version text -- which prompt version produced it
completed_at timestampz
```

scan_results

```
id uuid PK
scan_id uuid UNIQUE, references scans(id) on delete cascade
user_id uuid, references auth.users(id) on delete cascade
created_at
```

```

overall int, 0-100
clarity int, texture int, pores int, hydration int, redness int,
evenness int, firmness int  -- all 0-100
headline text not null
observations jsonb default '[]'
focus_areas jsonb default '[]'
refer_to_professional boolean default false
raw jsonb -- full model response, for debugging

routines
  id uuid PK
  user_id uuid, references auth.users(id) on delete cascade
  created_at
  source_scan_id uuid, references scans(id) on delete set null
  is_active boolean default true
  steps jsonb default '[]'

routine_logs
  id uuid PK
  user_id uuid, references auth.users(id) on delete cascade
  routine_id uuid, references routines(id) on delete cascade
  log_date date not null
  slot text, constrained to: morning, evening
  step_key text not null
  completed_at timestamptz default now()
  UNIQUE (user_id, log_date, slot, step_key)

ALSO INCLUDE
- Indexes on scans(user_id, created_at desc), scan_results(user_id, created_at),
  routine_logs(user_id, log_date).
- A trigger that inserts a profiles row automatically whenever a new auth.users
  row is created.
- A trigger that maintains profiles.updated_at.
- A partial unique index ensuring only one active routine per user.
- RLS policies on all five tables: select, insert, update and delete, each
  restricted to auth.uid() = user_id (or = id for profiles).
- A security-definer function `delete_my_account()` that deletes the calling
  user's rows across all tables and their storage objects, for GDPR/POPIA
  compliance. It must only ever act on auth.uid().

```

Write it as one idempotent SQL file I can paste into the Supabase SQL editor.
Add comments explaining each RLS policy in plain English.

Done when: the SQL runs without error in the Supabase SQL editor, and in the Table Editor every table shows RLS as enabled.

Verify RLS manually — do not take it on trust. In the SQL editor, run a query as an anonymous role against `scans`. It must return zero rows. A misconfigured policy here means every user's face photographs are readable by anyone with your public anon key, which is a data breach, a POPIA violation, and the end of the app.

Commit: `feat(db): initial schema with row-level security`

P3.3 — Private storage bucket

PASTE INTO CLAUDE CODE

Write a second migration, `supabase/migrations/0002_storage.sql`, that creates a storage bucket for face photographs.

Requirements:

- Bucket id: 'scans'
- PRIVATE (public = false). These are face photographs; they must never be reachable by URL without a signed token.
- A 6 MB file size limit.
- Allowed MIME types: image/jpeg, image/webp only.
- Storage RLS policies such that a user can only insert, select and delete objects whose path begins with their own user id. The path convention is: `{user_id}/{scan_id}.jpg`
- A policy allowing the service role full access, so the Edge Function can read images for analysis.

Add a comment block explaining the path convention and why the bucket must stay private.

Done when: the bucket exists, is marked Private, and a test upload from one account is not readable by another.

Commit: `feat(db): add private scans storage bucket`

P3.4 — Typed Supabase client

PASTE INTO CLAUDE CODE

Set up the Supabase client for the app.

1. Generate TypeScript types from the database schema into `src/types/database.ts`. Add an npm script "types:db" that regenerates them via the Supabase CLI, and document in the README that this must be re-run after every migration.
2. Create `src/lib/supabase.ts` exporting a typed singleton client:
 - Reads `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY`
 - Throws a clear, actionable error at startup if either is missing
 - Uses expo-secure-store as the auth storage adapter so sessions survive app restarts and are encrypted at rest
 - Sets `autoRefreshToken` true, `persistSession` true, `detectSessionInUrl` false
 - Handles React Native's AppState so token refresh pauses in the background
3. Create `src/lib/errors.ts` with a small helper that converts a Supabase or network error into a safe, user-facing message. It must never leak raw Postgres errors, table names, or policy details into the UI. Unknown errors map to a generic message from `copy.ts`. Log the real error to the console in development only.

Do not create any screens. Client setup only.

Done when: the app boots, and a quick temporary call to `supabase.auth.getSession()` returns without throwing.

Commit: `feat: add typed supabase client and error mapping`

9. Phase 4 — Auth and onboarding

Goal: the lowest-friction path from install to first scan. Every extra tap here costs you conversions, so the sign-up wall comes *after* the value, not before.

The onboarding order is a monetisation decision. Collect a little context first (it makes the result feel personalised and increases perceived value), then let them scan, then show the result, and only then ask for an account and money. Making someone create an account before they have seen anything is the single biggest install-to-activation leak in this category.

P4.1 — Anonymous-first authentication

PASTE INTO CLAUDE CODE

Implement authentication using Supabase Auth, anonymous-first.

FLOW

- On first launch, sign the user in ANONYMOUSLY. No screen, no friction. They get a real user id and a real profiles row immediately.
- They complete onboarding and their first scan as an anonymous user.
- We only prompt to create a real account at the point where it matters: when they subscribe, or when they tap "save my progress".
- Upgrading an anonymous account to a permanent one must preserve the SAME user id, so all their scans and history carry over.

BUILD

1. `src/stores/useAuthStore.ts` — a Zustand store holding session, user, profile, isLoading and isAnonymous. Actions: `initialise`, `signInAnonymously`, `linkEmail`, `signInWithEmail`, `signOut`, `refreshProfile`, `deleteAccount`.
2. Bootstrapping in `app/_layout.tsx`: on mount, restore any existing session; if there is none, sign in anonymously. Show a splash state until this resolves, so no screen ever renders without a user. Handle failure with a retry, not a crash.
3. Email sign-in via magic link (no passwords to forget or store). Screens at `app/(auth)/sign-in.tsx` and `app/(auth)/check-email.tsx`. Configure the deep link so the magic link returns into the app.
4. An `'upgradeAnonymousAccount(email)'` action that links the email identity to the existing anonymous user, preserving the user id.
5. A route guard in `app/_layout.tsx` that sends users to onboarding if `profiles.onboarding_completed_at` is null, and to the tabs otherwise.

RULES

- Never block the first launch on a network call longer than ~3 seconds; fall back to a retry state.
- All strings from `copy.ts`.
- Handle the case where anonymous sign-in fails entirely (offline first launch) with a clear retry screen.

Note: anonymous sign-in must be enabled in the Supabase dashboard. Tell me exactly where to switch it on.

Done when: a fresh install lands on onboarding with no sign-in screen, and the same user id survives an app restart.

Commit: `feat(auth): anonymous-first auth with email upgrade path`

P4.2 — Onboarding flow

PASTE INTO CLAUDE CODE

Build the onboarding flow at `app/(onboarding)/`. Five screens, swipeable, with a progress indicator, a back button, and no way to skip.

SCREENS

1. welcome — the promise in one line plus a single primary button. Set the tone: calm, honest, no hype. Mention in one quiet line that this gives cosmetic estimates, not medical advice.
2. age-band — single select, five bands from 18-24 to 55+.
3. skin-type — single select: oily, dry, combination, normal, sensitive, "I'm not sure". Each option needs one short plain-language descriptor underneath. "I'm not sure" must be a first-class option, not an apology — most people genuinely don't know.
4. concerns — multi-select chips, max 3 selections, with a counter. Use cosmetic language only: visible blemishes, uneven tone, visible pores, dryness, dullness, fine lines, redness, oiliness.
5. goal — single select: a primary outcome they care about, phrased as appearance goals.

Then a final "ready" screen that summarises what they chose in a friendly sentence and has one button: "Take my first scan".

BEHAVIOUR

- Persist each answer to the profiles table as they go, so quitting mid-flow doesn't lose progress and resuming returns them to the right step.
- Set `onboarding_completed_at` when they finish.
- Animate transitions between steps (slide, ~250ms).
- Haptic feedback on selection.
- The primary button is disabled until a valid selection exists.

RULES

- Under 18 is deliberately excluded from the age bands. If someone would fall outside the available options, we simply don't collect it — do not add an under-18 band.
- All strings from `copy.ts`. All colours and spacing from tokens.
- Each screen must work on a small phone (360dp wide) without scrolling if possible, and scroll gracefully if not.

Done when: you can complete onboarding on your phone, force-quit halfway and resume at the right step, and see the answers in the Supabase table editor.

Commit: `feat(onboarding): five-step onboarding flow with persistence`

Do not add an under-18 age band, and set your Play target audience to 18+. Collecting data from minors pulls you into Google Play's Families policy and a much stricter data regime, and face photographs of minors is a category of risk you do not want anywhere near a solo-run app. Keep the audience adult and say so in the store listing.

10. Phase 5 — Camera capture

Goal: consistent, comparable photographs. This phase matters more than it looks — if lighting varies wildly between scans, your scores will jump around, the progress graph will look like noise, and the core promise of the app breaks.

Consistency beats quality. A mediocre photo taken the same way every week produces a more useful trend than a beautiful photo taken differently each time. The guides, the lighting check and the ghost overlay all exist to force consistency, not to make the photo pretty.

P5.1 — Camera screen

PASTE INTO CLAUDE CODE

Build the capture screen at `app/scan/capture.tsx` using `expo-camera`.

LAYOUT

- Full-bleed front camera preview.
- A face-shaped oval overlay, centred, sized so a correctly-framed face fills it. Everything outside the oval is dimmed.
- A live guidance line above the shutter that shows the single most important correction right now, in priority order:
"Move to brighter light" > "Hold steady" > "Move a little closer" > "Move back a little" > "Center your face" > "Looking good"
- A large shutter button, disabled until guidance reads "Looking good".
- A small "why?" link opening a sheet explaining how to get comparable results: same room, same time of day, no makeup, hair pulled back, neutral expression.

CHECKS (run on the preview, roughly 2-3 times a second, not every frame)

1. Brightness — sample the preview and compute mean luminance. Flag too dark and too bright.
2. Face presence, size and centring — use `expo-face-detector` if it is available for the current SDK; if it is not, fall back to brightness-only checks and simply trust the oval guide. Tell me which path you took.
3. Stability — use `expo-sensors` accelerometer to detect excessive movement.

CAPTURE

- On shutter: haptic, capture at high quality, then immediately process with `expo-image-manipulator`: resize so the longest edge is 1024px, rotate to correct orientation, compress to JPEG at 0.85 quality.
- Show a confirm screen: the photo, "Use this" and "Retake". Do not upload before they confirm.
- Store the computed brightness/stability values to send along as `capture_quality`.

PERMISSIONS

- Request camera permission in context, with an explanation screen first, not a cold system prompt.
- Handle permanent denial with a screen that explains and deep-links to system settings.

RULES

- Front camera by default, with a flip toggle.
- Portrait only.
- Never write the photo to the device photo library.
- All strings from `copy.ts`.

Done when: on a real phone, the guidance text changes as you move into and out of good light, the shutter stays disabled until conditions are good, and the resulting file is about 1024px on its longest edge and comfortably under 500 KB.

Commit: `feat(scan): camera capture with framing and lighting guidance`

P5.2 — Ghost overlay for repeat scans

This is a small feature with a large effect on both data quality and perceived sophistication.

PASTE INTO CLAUDE CODE

Add a "ghost overlay" to the capture screen.

When the user has at least one previous scan, show their most recent scan photo as a faint (about 20% opacity) overlay on the camera preview, so they can line their face up in the same position as last time.

- A toggle to turn it on and off, defaulting to ON for repeat scans.
- A one-time tooltip the first time it appears, explaining that matching the previous position makes the before/after comparison far more accurate.
- Fetch the previous image via a short-lived signed URL, cache it locally, and never leave it on disk unencrypted longer than the session.

This directly improves both the accuracy of the trend data and the quality of the before/after slider, so it is worth doing properly.

Done when: on a second scan, a faint version of the previous photo appears over the preview and can be toggled off.

Commit: `feat(scan): add ghost overlay for consistent repeat framing`

11. Phase 6 — The AI scan pipeline

Goal: image in, validated scores out, with the API key never leaving the server and the model swappable in one line.

Architecture

APP	SUPABASE	GEMINI
---	-----	-----
1. compress image		
2. upload	-----> Storage (private bucket) scans/{user_id}/{scan_id}.jpg	
3. insert	-----> scans row, status = 'pending'	
4. invoke	-----> Edge Function: analyze-scan	
	 verify JWT, verify ownership set status = 'processing' download image (service role) base64 encode -----> generateContent (structured JSON) <----- JSON response validate against schema clamp + sanity-check scores insert scan_results set status = 'complete' 	
5. realtime	<----- scans row UPDATE	
6. navigate to results		

The Gemini key lives only in Supabase Edge Function secrets. If it is ever in the app bundle, in `.env` with an `EXPO_PUBLIC_` prefix, or in a committed file, anyone can pull it out of the APK and spend your money. The Edge Function exists for exactly this reason.

P6.1 — The Edge Function

PASTE INTO CLAUDE CODE

Create a Supabase Edge Function at `supabase/functions/analyze-scan/index.ts` (Deno + TypeScript).

CONTRACT

Input: POST { scanId: string }, with the user's JWT in the Authorization header

Output: 200 { ok: true } on success, or a 4xx/5xx with a machine-readable error code. The client gets results via Realtime, not this response.

STEPS

1. Validate the JWT. Reject anonymous-but-invalid or missing tokens with 401.
2. Load the scan row using the service role client. Verify scan.user_id matches the JWT's user id. Mismatch = 403. This check is mandatory – never trust the scanId alone.
3. If status is not 'pending', return 409. This makes the function idempotent and prevents double-billing from retries.
4. Set status = 'processing'.
5. Create a short-lived signed URL for the image, download it, base64 encode it. Reject anything over 6 MB.
6. Call Gemini generateContent with:
 - the system prompt from `./prompt.ts`
 - the image as `inline_data`
 - a responseSchema enforcing structured JSON output

- temperature 0.2 for consistency between scans
 - a 25-second timeout
7. Validate the response with Zod. If validation fails, retry ONCE with a stricter "return only valid JSON" instruction. If it fails again, set status = 'failed' with a reason and return.
 8. If the model returned usable = false, set status = 'rejected', store the reject reason, and return. Do not create a scan_results row.
 9. Clamp every score to 0-100 and integers. Recompute `overall` server-side as a weighted mean of the attributes rather than trusting the model's own figure, so it can never contradict them.
 10. Insert scan_results. Set scans.status = 'complete', completed_at, model and prompt_version.

ERROR HANDLING

- Every failure path must leave the scan row in a terminal state ('failed' or 'rejected'). A row stuck on 'processing' means a user staring at a spinner forever. Use a try/finally.
- Log errors with the scan id but NEVER log the image or any base64 data.
- Return structured error codes the client can map to copy.ts strings: UNAUTHORIZED, NOT_FOUND, ALREADY_PROCESSED, IMAGE_TOO_LARGE, MODEL_TIMEOUT, MODEL_INVALID_RESPONSE, INTERNAL.

MODEL ABSTRACTION

Put the model name, endpoint and pricing metadata in a single exported const in ./model.ts, so swapping model is a one-line change. Read the API key from Deno.env.get('GEMINI_API_KEY') and fail loudly at startup if it's absent.

Also write supabase/functions/analyze-scan/README.md documenting how to set secrets and deploy.

Done when: `supabase functions deploy analyze-scan` succeeds, and invoking it with another user's scan id returns 403.

Commit: `feat(api): add analyze-scan edge function`

P6.2 — The vision system prompt

This is the intellectual core of the product. Treat it as a versioned artefact: every change gets a new version string stored on the scan row, so you can always tell which prompt produced which historical result.

PASTE INTO CLAUDE CODE

Create supabase/functions/analyze-scan/prompt.ts exporting:

- SYSTEM_PROMPT (string)
- RESPONSE_SCHEMA (the Gemini responseSchema object)
- PROMPT_VERSION (string, start at "v1.0.0")

The system prompt must implement exactly the following. Write it out in full, clearly structured, no summarising:

ROLE

A cosmetic skin appearance analyser for a consumer beauty app. It describes the visible cosmetic appearance of skin in a photograph. It is a beauty tool, not a medical one.

ABSOLUTE RULES

1. Never diagnose. Never name any medical condition, disease or disorder.
2. Never mention or evaluate any treatment, medication or procedure.
3. Describe only what is visible in the photograph, in cosmetic terms.
4. Never comment on attractiveness, beauty, weight, age, gender, ethnicity, or any characteristic other than the cosmetic surface appearance of skin.
5. Never express alarm. Tone is calm, warm, specific and encouraging.
6. If something appears to warrant professional attention, do NOT describe it. Set refer_to_professional = true and say nothing further about it. The app

handles that message.

SCORING

- 0-100 per attribute, where 100 is the smoothest, most even appearance.
- Be calibrated, not generous. Most real photographs land between 55 and 85.
- Never return above 95 or below 25.
- Scores must be internally consistent with the written observations.

ATTRIBUTES (score each)

clarity	visible blemishes and marks on the surface
texture	smoothness and uniformity of the surface
pores	visibility of pores
hydration	how plump and dewy versus flat and tight the surface appears
redness	visible flushing or colour unevenness
evenness	uniformity of overall skin tone
firmness	visible tone and definition

IMAGE REJECTION

Set usable = false with a reject_reason from exactly this list when applicable:

no_face, multiple_faces, too_dark, too_bright, too_blurry, face_too_small, heavy_makeup, obstructed, not_a_photo

Do not score an unusable image.

OUTPUT

- headline: one warm, specific sentence summarising the overall appearance.
- observations: 2 to 4 short, specific, neutral observations. Each names an attribute and describes what is visible, in cosmetic language.
- focus_areas: 1 to 3 attribute keys the user would benefit most from focusing on, ordered by impact.

BANNED VOCABULARY (never output any of these)

diagnose, diagnosis, condition, disease, disorder, symptom, treat, treatment, cure, heal, therapy, medical, clinical, patient, prescription, lesion, acne vulgaris, rosacea, eczema, psoriasis, dermatitis, melasma

REQUIRED VOCABULARY

"the appearance of", "looks", "visible", "cosmetic"
Address the user as "your skin", never "the subject".

RESPONSE_SCHEMA must enforce: usable (boolean), reject_reason (string|null), overall (integer), the seven attribute integers, headline (string), observations (array of strings), focus_areas (array of strings), refer_to_professional (boolean). All required.

Add a comment block at the top explaining that PROMPT_VERSION must be bumped on every change, because it is stored against each scan row for reproducibility.

Done when: the file exists and you have read the full prompt against the banned-word list yourself.

Commit: `feat(api): add versioned vision system prompt`

P6.3 — Calibration harness

Do not skip this. Without it you are shipping a scoring system you have never actually evaluated.

PASTE INTO CLAUDE CODE

Create a small local calibration harness at `scripts/calibrate.ts`.

It should:

1. Read every image in `scripts/fixtures/` (I will supply 15-20 face photos: varied skin tones, lighting conditions, ages, and some deliberately bad ones – too dark, blurry, no face, two faces).
2. Run each through the same prompt and model the Edge Function uses.
3. Run each image THREE times, so I can measure consistency.
4. Write `scripts/calibration-report.md` containing, per image:
 - all three score sets side by side
 - the variance across the three runs, per attribute
 - the headline and observations from run one
 - a compliance check flagging any banned vocabulary in the output
5. Print a summary: mean variance per attribute, the overall score distribution, and any images where the three runs disagreed by more than 8 points on any attribute.

This must be runnable from my machine with my own API key in a local `.env` that is gitignored. Do not commit fixtures containing real faces.

Done when: the report runs and no attribute has a mean run-to-run variance above about 5 points.

This is the acceptance gate for the whole product. If the same photograph scores 72, 61 and 80 across three runs, your progress graph is measuring model noise, not skin. Fix it before building anything on top: lower the temperature, make the scoring rubric in the prompt more concrete and anchored, or move up to the next model tier. A tracker built on an inconsistent scorer is worthless, and users will notice within three scans.

Commit: `test: add model calibration harness`

P6.4 — Wire the client to the pipeline

PASTE INTO CLAUDE CODE

Connect the app to the scan pipeline.

BUILD

1. `src/lib/scan.ts` with a ``submitScan(imageUri, captureQuality)`` function that:
 - generates a scan id client-side (uuid)
 - uploads the image to `scans/{user_id}/{scan_id}.jpg`
 - inserts the scans row with status `'pending'` and the `capture_quality` data
 - invokes the `analyze-scan` Edge Function
 - returns the scan id

Each step must have its own error handling; a failed upload must not leave an orphan database row, and a failed insert must not leave an orphan file.
2. `src/stores/useScanStore.ts` – Zustand store for the in-flight scan: current scan id, status, result, error. Subscribe to Supabase Realtime on the scans row and update as status changes.
3. `app/scan/analysing.tsx` – the waiting screen. This is a moment of anticipation, so make it feel considered:
 - the captured photo, subtly animated (a slow scanning sweep)
 - rotating status lines that reflect real progress, not fake progress
 - a progress indication that does not lie: indeterminate until we know more
 - after 30 seconds, offer a "still working" message; after 60, offer to retry or go back
 - on `'complete'`, navigate to results
 - on `'rejected'`, show the specific reason from `copy.ts` with a "retake" button
 - on `'failed'`, show an error with retry that does NOT consume their free scan

4. A safety net: if Realtime disconnects, fall back to polling the scan row every 3 seconds. The user must never be stranded on this screen.

RULES

- The free scan counter (`profiles.free_scan_used`) must only be set on a `SUCCESSFUL`, usable scan. Never burn someone's free scan on a rejected photo or a server error – this is the fastest way to a one-star review.

Done when: you can scan on a real phone and watch a `scan_results` row appear in Supabase, and a deliberately bad photo (lens covered) returns a clean "retake" message without burning the free scan.

Commit: `feat(scan): wire client to analysis pipeline with realtime updates`

12. Phase 7 — The results screen

Goal: the moment that makes people screenshot. This screen is your marketing asset, your conversion driver and your retention hook, so it deserves disproportionate effort.

This screen is the product's advertisement. Every viral app in this category grew because the reveal looked good on camera. Build it so that someone filming their phone gets a satisfying three seconds. Staged animation, a real sense of arrival, no clutter.

P7.1 — Results screen

PASTE INTO CLAUDE CODE

Build the results screen at `app/scan/result.tsx`.

STRUCTURE, in reveal order

1. The overall score, large, centre-stage, in a `ScoreRing` that sweeps from 0 and counts up over about 900ms. Haptic tap on arrival.
2. The headline sentence, fading in after the ring lands.
3. A grid of the seven attribute scores as smaller rings or bars, animating in with a 60ms stagger between them.
4. Observations – each as its own card, appearing in sequence.
5. Focus areas – the 1-3 attributes to prioritise, visually distinct.
6. If this is not their first scan: a "change since last scan" row showing the delta per attribute with up/down indicators and the number of days between scans.
7. The Disclaimer component, always visible, not collapsible.
8. If `refer_to_professional` is true: a calm, quiet card using `copy.disclaimers.referral`. It must not describe anything, must not be red, and must not use alarm language.
9. Primary action: "Build my routine" (first scan) or "See my progress" (repeat scan).

ANIMATION

- Use `react-native-reanimated`. Total reveal under 2.5 seconds.
- A "skip" affordance: tapping anywhere completes all animations instantly. Never trap someone in an animation.
- Respect the OS reduce-motion setting – if enabled, render everything immediately with no motion.

RULES

- All colours from the tokens ``score`` scale. A low score must never look like a failure or an error – this is the emotional core of the app and shaming people is both cruel and bad business. Low scores are "room to improve", rendered in a neutral-warm tone, never red.
- The screen must be fully readable without any animation having run.
- All strings from `copy.ts`.
- Works on a 360dp-wide screen.

Done when: the reveal feels good on a real device, tapping skips the animation, and a deliberately low-scoring result still reads as encouraging rather than damning.

Commit: `feat(results): animated results reveal screen`

P7.2 — Shareable progress card (build this, even though it is v1.1)

PASTE INTO CLAUDE CODE

Build a shareable result card.

Add a "Share" action to the results screen that generates an image and opens the native share sheet.

THE CARD

- Rendered off-screen with react-native-view-shot, exported at 1080x1920 (story format) and 1080x1350 (feed format), user picks or defaults to story.
- Contains: the overall score in a large ring, two or three attribute scores, the date, a subtle app name/logo, and for repeat scans a before/after score comparison.
- IMPORTANT: it must NOT include the user's face photograph by default. Offer a toggle for that, defaulting to OFF. Sharing your face by accident is a privacy failure.
- Design it to look good as a screenshot: high contrast, large type, no thin hairlines, nothing that relies on colour alone.

Track a "share_initiated" analytics event with the surface and format.

Done when: the share sheet opens with a clean card image, and the face photo is excluded unless explicitly enabled.

Commit: `feat(results): add shareable progress card`

13. Phase 8 — Progress tracking

Goal: the actual product. Everything before this is the hook; this is the reason someone still pays in month three.

P8.1 — Progress screen

PASTE INTO CLAUDE CODE

Build the progress screen at `app/(tabs)/progress.tsx`.

SECTIONS

1. Hero: the current overall score, and the change since the first scan, framed as a journey ("up 8 points in 6 weeks"). If there is only one scan, show an encouraging empty-ish state explaining that the second scan is where this gets interesting, with the date of their next suggested scan.
2. Trend chart: overall score over time. Built with `react-native-svg` — do not add a charting library. Requirements:
 - x axis by date, handling irregular intervals correctly (do not treat scans as evenly spaced)
 - a smoothed line plus the actual data points
 - tap a point to see that scan's details
 - a subtle band showing the expected noise range, so a 2-point wobble does not read as a real change
 - handles 2 points and 50 points equally gracefully
3. Attribute filter: chips to switch the chart between overall and each of the seven attributes.
4. Scan history: a reverse-chronological list. Each row shows the thumbnail, date, overall score and the delta from the previous scan. Tapping opens that scan's result screen.
5. Streak: consecutive weeks with at least one scan. Present it warmly and never punitively — a broken streak gets "let's start a new one", not a guilt trip.

RULES

- Every section needs a real empty state for a user with one scan.
- Thumbnails load via short-lived signed URLs, cached in memory for the session.
- Never render a chart with a misleading y-axis. Either start at 0 or clearly indicate the range.

Done when: with three scans in the database, the chart renders correctly, respects irregular date gaps, and every attribute filter works.

Commit: `feat(progress): trend chart and scan history`

P8.2 — Before and after comparison

PASTE INTO CLAUDE CODE

Build the before/after comparison at `app/progress/compare.tsx`.

FEATURES

- Two scan pickers: "before" defaults to the first scan, "after" defaults to the most recent. Both changeable from a list of all scans.
- A draggable slider that wipes between the two photographs. Smooth, 60fps, built with `react-native-gesture-handler` and `reanimated`.
- Below the images: a side-by-side score comparison with the delta per attribute, and the elapsed time between the two scans.
- A share action that exports the comparison as an image. The same privacy rule applies: face photos included only with explicit opt-in, defaulting to off, with a clear confirmation the first time.

RULES

- Both images load via signed URLs; handle expiry by refetching.
- If the two photos have different aspect ratios, crop to a common frame rather than distorting either.
- Handle the case of fewer than two scans with an empty state.

Done when: the slider is smooth on a mid-range Android phone and the score deltas match the database.

Commit: `feat(progress): before/after comparison slider`

14. Phase 9 — Routine and reminders

Goal: give the user something to do between scans, and a reason to come back. Daily engagement between weekly scans is what turns a novelty into a habit.

P9.1 — Routine generation and display

PASTE INTO CLAUDE CODE

Build the routine feature.

GENERATION

Extend the analyze-scan Edge Function to also generate a simple routine, in the SAME model call (do not make a second API call – it doubles cost and latency).

The routine is:

- 3 to 5 morning steps, 3 to 5 evening steps
- Each step: a key, a short title, a one-line "why", and a time-of-day slot
- Generic product CATEGORIES only, never brand names and never active ingredient dosages. "A gentle cleanser", not "salicylic acid 2%".
- Ordered correctly (cleanser, then treat, then moisturise, then SPF)
- Sunscreen is always the final morning step, without exception

Add this to the response schema and store it in the routines table with source_scan_id set.

COMPLIANCE

The routine must read as cosmetic self-care guidance, not as instruction. No dosages, no actives by name, no frequency prescriptions, no claims about what it will fix. Phrase as "many people find..." rather than "you should...".

DISPLAY – app/(tabs)/routine.tsx

- Two sections: Morning and Evening, with the current one expanded by default based on the device clock.
- Each step is a tappable row with a checkbox, title and "why" line.
- Ticking a step writes a routine_logs row and gives haptic feedback.
- A daily completion ring at the top.
- Resets at local midnight in the user's timezone, not UTC.
- A gentle line noting that consistency over weeks is what shows up in scans.

RULES

- Optimistic UI on tick, with rollback on failure.
- Works offline: queue writes locally and sync when connectivity returns.
- Empty state before their first scan explains that the routine appears after their first scan.

Done when: a scan produces a routine, ticking steps persists across restarts, and the day rolls over correctly at local midnight.

Commit: `feat(routine): routine generation and daily tracking`

P9.2 — Notifications

PASTE INTO CLAUDE CODE

Implement local notifications with expo-notifications. Local only – no push server, no remote notifications.

NOTIFICATIONS

1. Weekly re-scan reminder – the single most important retention mechanism. Scheduled for the user's chosen weekday and hour, 7 days after their last scan. Copy should be specific and curious, never nagging. Vary the message across a small set so it doesn't feel robotic.
2. Daily routine reminders – morning and evening, both opt-in and both defaulting to OFF. Only offer these after they have used the routine tab at least twice.
3. Streak-at-risk – fires only if they have a streak of 2+ weeks and are one day from breaking it. Warm, not guilt-tripping. Hard cap at one per week.

PERMISSIONS

- Ask AFTER their first successful scan, never on launch, and only with an in-app explanation screen first that states what we'll send and how often.
- If denied, never ask again via the system prompt. Offer a settings route.

CONTROLS

Full control in settings: a master switch, per-type switches, and day/time pickers for the weekly reminder. Changes reschedule immediately.

RULES

- Cancel and reschedule the weekly reminder after every completed scan.
- Never send more than one notification per day under any circumstances.
- Respect the device's do-not-disturb.
- All copy from copy.ts.

Done when: a test notification fires at a scheduled time, all toggles work, and completing a scan reschedules the weekly reminder.

Commit: feat: local notifications for re-scan and routine

15. Phase 10 — Paywall and subscriptions

Goal: revenue, without the dark patterns that get apps pulled.

Apple pulled a major app in this exact category in April 2026 for a paywall that displayed the weekly-equivalent price more prominently than the amount actually billed, and obscured auto-renewal. Google enforces comparably. Show the real billed amount, in the same visual weight as everything else, and state the renewal terms plainly. An honest paywall converts nearly as well and does not put your entire business at the mercy of one policy review.

Pricing structure

Plan	Price (USD tier-1)	Price (ZAR)	Role
Weekly + 3-day trial	\$4.99	~R89	The conversion workhorse. Low commitment, trial removes risk
Annual	\$39.99	~R719	The value anchor and the LTV driver
Monthly	\$9.99	~R179	Decoy. Makes annual look obviously correct. Optional

Structure: hard paywall after one free scan. Hard paywalls convert several times better than freemium in this category, and the first scan delivers enough value to justify asking.

Regional pricing: set the USD figures for the US, UK, EU, Canada and Australia, where the money is. Then price down materially for South Africa, India, Brazil, Southeast Asia and similar markets — something like R49/week and R399/year — or you will convert almost nobody there. Both stores support per-country pricing; use it.

On your original R100/month idea. R100 (~\$5.50) is a perfectly reasonable monthly price, but as the headline plan it underprices the category and misses how it actually monetises. The weekly-plus-trial structure captures impulse conversions at the moment of peak interest, the annual plan captures committed users at a much higher lifetime value, and monthly mostly exists to make annual look cheap. Keep a monthly tier if you like — just don't make it the star.

P10.1 — Store products and RevenueCat

Manual configuration, in this order. Do this before writing the paywall code.

1. **Play Console** > Monetise > Subscriptions. Create one subscription product, **glowtrack_premium**, with three base plans: **weekly**, **monthly**, **annual**.
2. On the weekly base plan, add a **free trial offer** of 3 days.
3. Set prices for your tier-1 markets, then override for your discounted regions.
4. **RevenueCat**: create a project, add the Android app, and upload the Google Play service account JSON credentials.
5. Create an entitlement called **premium** and attach all three base plans to it.
6. Create an offering called **default** with three packages: weekly, monthly, annual.

Google Play subscription products can take several hours to propagate and will not appear in a debug build at all. You must test purchases from a build uploaded to a Play testing track, signed with the same key, with a licence-tested account. Budget a full day the first time — nothing about this works the way you expect.

P10.2 — Paywall implementation

PASTE INTO CLAUDE CODE

Implement subscriptions with RevenueCat.

SETUP

1. Install react-native-purchases. Configure it in app/_layout.tsx after auth resolves, using the RevenueCat user id set to the Supabase user id so the two systems stay linked across reinstalls.
2. src/lib/purchases.ts wrapping the SDK: getOfferings, purchasePackage, restorePurchases, and a getCustomerInfo that resolves entitlement state.
3. src/stores/useSubscriptionStore.ts holding isPremium, offerings, isLoading. Refresh entitlement state on app foreground.

PAYWALL — app/paywall.tsx

Structure:

- A headline focused on the progress promise, not on features.
- Three or four short value lines, each concrete.
- Three plan cards. Annual is pre-selected and carries a "best value" badge showing the honest percentage saving versus weekly.
- For the weekly plan with a trial, the button reads "Start 3-day free trial" and DIRECTLY BELOW IT, in normal body text at normal weight, the exact terms: what they will be charged, when, and that it renews until cancelled.
- A "Restore purchases" link. This is mandatory for store approval.
- Links to Terms and Privacy Policy. Also mandatory.
- A close button that is always visible and never delayed.

HONESTY RULES — non-negotiable

- Never show a weekly-equivalent price more prominently than the actual billed amount.
- Never use a fake countdown, fake scarcity, or a fake discount.
- Never make the close button hard to find or delay its appearance.
- The exact billed amount and renewal terms appear before the purchase button, not behind a link.
- Use the localised price string from RevenueCat, never a hardcoded price.

GATING

- The first scan is free. After it, profiles.free_scan_used is true and any further scan attempt routes to the paywall.
- Results from the free scan remain viewable forever. Never take away something they have already been shown.
- Progress and comparison features require premium.
- A premium user who lapses keeps read access to their history but cannot scan.

RULES

- Handle every purchase outcome: success, user cancelled, payment pending, already subscribed, store unavailable, network failure.
- Never trust the client for entitlement on anything sensitive; the Edge Function should check entitlement before spending money on a scan.
- Track paywall_viewed, plan_selected, purchase_started, purchase_completed, purchase_failed and purchase_cancelled, each with the plan id.

Done when: a real purchase completes from a Play testing track using a licence-tested account, the entitlement unlocks scanning, and restore works on a fresh install.

Commit: feat: revenuecat subscriptions and paywall

P10.3 — Server-side entitlement check

PASTE INTO CLAUDE CODE

Add an entitlement check to the analyze-scan Edge Function.

Before spending anything on a model call:

1. If `profiles.free_scan_used` is false, allow the scan and mark it used only AFTER a successful, usable result.
2. Otherwise verify an active premium entitlement by calling the RevenueCat REST API with the user id, using an API key held in Edge Function secrets.
3. No entitlement = return 402 with code `SUBSCRIPTION_REQUIRED`. The client maps that to the paywall.
4. Cache the entitlement result for 5 minutes per user to avoid a RevenueCat call on every scan.

This matters because a modified client could otherwise call the function directly and run up your model bill indefinitely.

Done when: invoking the function directly without an entitlement returns 402.

Commit: `feat(api): server-side entitlement enforcement`

16. Phase 11 — Analytics and instrumentation

Goal: know why people leave. Without this you are guessing, and at your scale guessing is fatal.

P11.1 — Event instrumentation

PASTE INTO CLAUDE CODE

Set up PostHog analytics and Sentry crash reporting.

1. `src/lib/analytics.ts` wrapping PostHog with a typed `track()` function. Define an `EventName` union type so a typo becomes a compile error rather than a silently missing event.
2. Identify users by their Supabase user id. Set person properties: `skin_type`, `age_band`, `concerns count`, `is_premium`, `scan_count`, `days_since_install`.
3. Instrument exactly these events with the listed properties:

```

app_opened           { is_first_open, days_since_install }
onboarding_started   {}
onboarding_step      { step_name, step_index }
onboarding_completed { skin_type, age_band, concern_count }
camera_opened        { scan_number }
capture_attempted    { brightness_ok, face_ok, stability_ok }
scan_submitted       { scan_number, capture_quality }
scan_completed       { scan_number, overall_score, seconds_elapsed }
scan_rejected        { reject_reason, scan_number }
scan_failed          { error_code }
results_viewed       { scan_number, overall_score }
share_initiated      { surface, format }
paywall_viewed       { trigger, scan_number }
plan_selected        { plan_id }
purchase_completed   { plan_id, price, currency, is_trial }
purchase_failed      { plan_id, reason }
routine_step_ticked  { slot, step_key, day_number }
progress_viewed      { scan_count }
comparison_viewed    { days_between }
rescan_from_reminder { days_since_last }
notification_permission { granted }

```

4. **PRIVACY** – never send: the image, the image path, any signed URL, the email address, the headline, or the observation text. Scores and counts only. Disable session recording and autocapture entirely.
5. Sentry: initialise with the DSN, enable it only in production builds, scrub all request bodies, and never attach image data to an event.
6. Build these four funnels in PostHog and note them in the README:
 - install -> onboarding_completed -> scan_completed (activation)
 - scan_completed -> paywall_viewed -> purchase_completed (monetisation)
 - scan_completed -> rescan (day 7) -> rescan (day 14) (the retention loop)
 - camera_opened -> capture_attempted -> scan_completed (capture funnel)

Done when: events appear in PostHog from a real device and the four funnels render.

Commit: `feat: analytics instrumentation and crash reporting`

The capture funnel is the one nobody instruments and everybody needs. If a large share of users open the camera and never successfully submit, your lighting checks are too strict, your guidance is unclear, or the shutter gating is frustrating people into quitting. That is a silent, invisible killer of activation, and only this funnel exposes it.

17. Phase 12 — Legal, privacy and compliance

Goal: survive review, and be genuinely defensible. You are storing face photographs, which is biometric-adjacent personal data under both POPIA and GDPR. Take it seriously.

The compliance position

Your defensive posture rests on four pillars. Each must hold.

1. **You are a Beauty app, not a Health or Medical app.** List under Beauty. Never use the banned vocabulary. Never imply diagnosis or treatment.
2. **You make no accuracy claims.** Never state a percentage accuracy, never say "dermatologist-grade", never compare yourself to a professional assessment.
3. **Disclaimers are prominent and everywhere a score appears.** Not buried in settings, not behind a link.
4. **Data handling is explicit, minimal and reversible.** Face photos are private, users can delete everything, and you say exactly what happens to the images.

P12.1 — In-app compliance surfaces

PASTE INTO CLAUDE CODE

Add the compliance and privacy surfaces to the app.

1. SETTINGS SCREEN — app/(tabs)/settings.tsx with sections:
 - Account: email if linked, or a "save my account" prompt if anonymous
 - Subscription: current plan, renewal date, a "manage subscription" link that opens the Play subscription page
 - Reminders: all notification controls
 - Your data:
 - "Download my data" — exports scans, scores and routines as JSON via the share sheet
 - "Delete all my scan photos" — deletes storage objects but keeps scores, with a clear explanation of what this does and doesn't remove
 - "Delete my account" — the full destructive path (below)
 - About: Privacy Policy, Terms, the full disclaimer, app version, a support email link
2. ACCOUNT DELETION — this is a store requirement, not optional.
 - A confirmation screen spelling out exactly what is destroyed
 - Requires typing DELETE to confirm
 - Calls the delete_my_account() database function
 - Deletes every storage object under their user id
 - Signs them out and returns to a fresh first-launch state
 - Must work for anonymous users too
3. FIRST-RUN DATA NOTICE — before the first camera use, a screen that states plainly, in the app's own voice, not legalese:
 - the photo is uploaded and analysed
 - it is stored privately so you can compare later
 - it is never shared, never sold, never used for advertising
 - you can delete it any time
 With "Continue" and "Not now" options.
4. The Disclaimer component must appear on: every results screen, the progress screen, the routine screen, and in settings.

RULES

- All strings from copy.ts.
- The delete flow must be genuinely destructive. Do not soft-delete and claim

deletion – that is a POPIA and GDPR problem, and a trust problem.

Done when: account deletion actually removes every row and every storage object, verified in the Supabase dashboard.

Commit: `feat: privacy controls, data export and account deletion`

P12.2 — Privacy policy and terms

PASTE INTO CLAUDE CODE

Draft a privacy policy and terms of service for the app as two markdown files in docs/. I will have these reviewed before publishing – produce a solid, specific starting point, not generic boilerplate.

CONTEXT

- Consumer mobile app, Android first
- Operated from South Africa, users worldwide
- Must address POPIA (South Africa) and GDPR (EU/UK) at minimum
- Stores face photographs, which are personal information and in some jurisdictions may be treated as biometric data

THE PRIVACY POLICY MUST COVER

- Exactly what is collected: face photographs, cosmetic appearance scores, skin type and concerns from onboarding, device and usage analytics, subscription status
- What is NOT collected: name, address, phone number, precise location, contacts, health records
- How photographs are used: sent to a third-party AI provider (Google) for cosmetic analysis, stored privately for the user's own comparison
- That photographs are NOT used to train any model, NOT shared with advertisers, NOT sold
- Third-party processors named individually: Supabase (storage and database), Google (AI analysis), RevenueCat (subscriptions), PostHog (analytics), Sentry (crash reporting)
- Retention: how long data is kept and what happens on deletion
- User rights under POPIA and GDPR: access, correction, deletion, portability, objection – and the exact in-app path to exercise each
- International transfer of data, since processors are outside South Africa
- Age restriction: 18+, and that the service is not directed at children
- Contact details for data requests
- Effective date and a change-notification process

THE TERMS MUST COVER

- The service provides COSMETIC APPEARANCE ESTIMATES ONLY, is not medical advice, is not a diagnosis, and is not a substitute for a qualified professional. State this prominently, near the top.
- That results are estimates affected by lighting, camera and image quality
- No warranty as to accuracy, and no guarantee of any outcome
- Subscription terms: billing, auto-renewal, the free trial, and that cancellation is handled through the platform store, not by us
- Acceptable use: own photographs only, no uploading images of other people without consent, no minors
- Limitation of liability
- Governing law

WRITE IN PLAIN ENGLISH. Short sentences. Where a legal term is unavoidable, explain it. A user should be able to read this and actually understand what happens to their face.

Done when: both documents are published at live, publicly reachable HTTPS URLs — as web pages, not PDFs — and linked from both the app and the store listing.

Have these reviewed by someone qualified before launch. A generated draft is a good starting point and a bad final document. You are handling face photographs across jurisdictions; a few hours of professional review is cheap insurance against a POPIA complaint or a GDPR data-subject request you handle wrongly.

Commit: docs: privacy policy and terms of service

P12.3 — Compliance audit

PASTE INTO CLAUDE CODE

Run a compliance audit across the entire codebase and report findings. Do not fix anything yet – produce the report first.

Check for and list every occurrence of:

1. BANNED VOCABULARY in any user-facing string, in copy.ts, in any component, in the Edge Function prompt, in app.json, or in README/store copy: diagnose, diagnosis, condition, disease, disorder, symptom, treat, treatment, cure, heal, therapy, medical, clinical, patient, prescription, lesion, dermatologist, acne vulgaris, rosacea, eczema, psoriasis, dermatitis, melasma
2. ACCURACY CLAIMS: any percentage, any "accurate", "precise", "proven", "scientifically", "clinically", or any comparison to a professional.
3. HARDCODED USER-FACING STRINGS outside copy.ts.
4. HARDCODED COLOURS, spacing values or font sizes outside tokens.ts.
5. SECRETS: any string matching an API key pattern anywhere in the repo, including git history. Check for AIza, sk-, eyJ, service_role, and any non-EXPO_PUBLIC env var referenced in app code.
6. MISSING DISCLAIMERS: any screen rendering a score without the Disclaimer component.
7. PERMISSIONS declared in app.json that the app does not actually use.

Output a markdown report at docs/compliance-audit.md with file, line, the issue and a suggested fix. Then wait for me to review it before changing anything.

Done when: the report shows zero findings in categories 1, 2, 5 and 6.

Commit: chore: compliance audit clean

18. Phase 13 — Quality assurance

P13.1 — Hardening pass

PASTE INTO CLAUDE CODE

Do a robustness pass across the app. For each of the scenarios below, either confirm it is already handled or fix it. Report what you found.

NETWORK

- Airplane mode on launch (before any session exists)
- Connection lost mid-upload
- Connection lost while waiting on results
- Very slow connection: does anything time out with a useful message?
- Supabase unreachable entirely

AUTH

- Session expired while the app was backgrounded for days
- Anonymous sign-in fails on first launch
- Magic link opened on a different device
- Account deleted server-side while the app is open

SCAN

- Camera permission denied, then granted later from settings
- Camera permission permanently denied
- App backgrounded mid-capture
- App killed during upload – does an orphan row or orphan file survive?
- Model returns malformed JSON
- Model times out
- Same scan submitted twice
- Storage quota exceeded

SUBSCRIPTION

- Purchase cancelled mid-flow
- Purchase pending (some payment methods take hours)
- Subscription expires while the app is open
- Restore with no prior purchases
- Play Store unavailable or signed out

DATA

- User with zero scans on every screen
- User with exactly one scan on the progress and compare screens
- User with 100 scans: does the history list stay smooth?
- Very long strings from the model breaking layout
- A scan row stuck in 'processing' from a previous crash

DEVICE

- Smallest common Android screen (360dp wide)
- Largest system font size
- Dark mode on every screen
- Device rotation while locked to portrait
- Low storage

For each fix, keep the change minimal and tell me what you changed.

Done when: every scenario above has been tested on a real device and behaves sensibly.

Commit: fix: robustness and edge case handling

P13.2 — Manual test script

PASTE INTO CLAUDE CODE

Write docs/test-script.md – a manual regression checklist I run before every release build.

Organise it as numbered steps with explicit expected results, grouped into:

- Fresh install and onboarding
- First scan (happy path)
- Bad photo handling (dark, blurry, no face, two faces)
- Results and sharing
- Paywall and purchase
- Second scan and comparison
- Routine and notifications
- Settings, data export and account deletion
- Offline behaviour

It must be followable by someone who has never seen the app, take under 30 minutes, and every step must state what "pass" looks like.

Commit: docs: manual regression test script

19. Phase 14 — Google Play and the 14-day gate

This phase runs in parallel with Phases 8-13. Start it in week 3.

The rule, precisely

A personal Play developer account created after 13 November 2023 must run a **closed test** with at least **12 testers opted in continuously for 14 days** before it can apply for production access. Google also assesses whether those testers genuinely used the app. Organisation accounts are exempt. Internal and open testing tracks do not count — only closed testing.

Meeting the threshold makes you *eligible to apply*. Google then reviews the application, typically within about seven days, and can ask for more.

A tester who opts out and rejoins restarts their own 14-day clock from zero. The earlier days do not bank. This is why you over-recruit to 15 and why you tell testers explicitly: install it, keep it installed, open it a few times a week, and do not leave the test.

P14.1 — First EAS build

PASTE INTO CLAUDE CODE

Set up EAS Build for Android.

1. Install eas-cli and run `eas build:configure`.
2. Create eas.json with three profiles:
 - development: development client, internal distribution, for debugging
 - preview: APK, internal distribution, for sharing a direct download
 - production: AAB (Android App Bundle), for Play Store submission
3. Configure app.json with: the package name, a versionCode, a version string, the adaptive icon, the splash screen, and permissions limited strictly to CAMERA and INTERNET. Remove anything else.
4. Set up EAS environment variables/secrets for the production profile so the EXPO_PUBLIC_* values are injected at build time and never committed.
5. Add npm scripts: "build:preview" and "build:production".
6. Explain to me, step by step, what happens with the Android signing keystore, why losing it means never being able to update the app again, and exactly how to back it up.

Then tell me the exact command to produce my first production AAB.

Done when: `eas build --profile production --platform android` produces a downloadable `.aab`, and you have the keystore backed up in two separate places.

Back up your Android keystore immediately, in two places, one of them offline. If you lose it, you can never publish an update to that listing again — ever. You would have to ship an entirely new app and abandon every install, review and subscriber. EAS manages it for you by default; download a copy anyway with `eas credentials`.

Commit: `build: configure EAS build profiles`

P14.2 — Play Console setup

Manual work. Allow a full working day.

1. **Create the app** in Play Console. Name, default language, app-or-game, free-or-paid (choose **Free** — the subscription is an in-app purchase).
2. **App content** — work through every item on this page. Google will not let you publish with anything outstanding:
 - Privacy policy URL (live web page, not a PDF)
 - App access: provide test credentials if anything is behind a login. Anonymous-first auth means most of your app is reachable, but note that scanning requires a subscription and explain how a reviewer can get past it
 - Ads: declare none
 - Content rating questionnaire
 - Target audience: **18 and over**
 - News app: no
 - Data safety: this is the long one. Declare photos, approximate device data, app activity. State that data is encrypted in transit, that users can request deletion, and describe the purpose of each item. **It must match your privacy policy exactly** — mismatches are a common rejection
 - Government apps: no
 - Financial features: none
 - Health apps declaration: if your Beauty categorisation is accepted you should not be prompted. If you are, answer honestly and lean hard on the cosmetic-not-medical framing
3. **Store listing**: app name, short description (80 characters), full description (4,000), screenshots, feature graphic, app icon.
4. **Category: Beauty**. Not Health & Fitness. Not Medical.
5. Upload the AAB to the **closed testing** track.
6. Create an email list of your testers and generate the opt-in link.

P14.3 — Store listing copy

PASTE INTO CLAUDE CODE

Write the Google Play store listing copy. Output to docs/store-listing.md.

DELIVERABLES

1. App title — max 30 characters. Must include the app name and hint at the core benefit.
2. Short description — max 80 characters. This is what people see in search results and it drives install rate more than anything else on the page. Give me five options.
3. Full description — max 4,000 characters. Structure:
 - a strong first two lines (all that shows before "read more")
 - the problem, in the user's own words
 - how it works, in three steps
 - what they get
 - a clear, unmissable cosmetic-not-medical disclaimer paragraph
 - subscription terms stated plainly
4. Screenshot plan — eight screenshots, with the caption text for each and a note on what the screen should show. Lead with the results reveal and the before/after comparison, because those are the two things that sell this.
5. Feature graphic concept — 1024x500, described.

COMPLIANCE — this copy is reviewed by a human at Google

- No banned vocabulary. None.
- No accuracy claims, no percentages, no "dermatologist" anything.
- No before/after claims implying a guaranteed outcome.
- No comparison to professional assessment.

- Subscription pricing and auto-renewal stated plainly.

TONE

Confident, plain, specific. Not hyped. Gender-neutral throughout – the copy must read equally naturally to a 19-year-old man and a 45-year-old woman.

Commit: docs: play store listing copy

P14.4 — Running the closed test

A practical sequence:

1. Upload the AAB to closed testing. Wait for review (usually hours to a couple of days for the first submission).
2. Create the tester email list. Send the opt-in link with a short, clear message.
3. **Tell testers exactly what to do.** Most failed closed tests fail because testers did not understand the ask:

What I need from you (takes 2 minutes to set up):

1. Tap this link on your Android phone and accept the invite
2. Install the app from the Play Store link that appears
3. Keep it installed for 14 days -- this is the important bit
4. Open it a few times a week and try a scan
5. Tell me anything that is broken, confusing or ugly

Please don't leave the test before [DATE] -- if you do, my 14-day clock resets and I have to start over. If you need to stop, just tell me first.

4. **Track opt-ins daily.** Play Console shows the count. If someone drops, replace them immediately — and remember the replacement starts at day zero.
5. **Keep shipping updates to the closed track.** Google explicitly views ongoing testing and iteration favourably, and it is the whole point of the exercise.
6. **Collect feedback in one place.** A WhatsApp group works well and testers actually use it.
7. **On day 14,** apply for production access on the Play Console dashboard. The application asks about your testing process, the feedback you received, and what you changed as a result. Answer specifically — vague answers get rejected and you reapply from the back of the queue.

Write the production-access application as you go. Keep a running note of feedback received and changes made during the test. On day 14 you will have a specific, credible answer instead of trying to reconstruct two weeks from memory.

20. Phase 15 — iOS, later, without a Mac

You can build and submit an iOS app entirely from Windows using EAS. What you cannot do without an iPhone is *verify* it. Do this only after Android has validated the product.

Step	Feasible from Windows?	Notes
Write the code	Yes	Same codebase
Build the IPA	Yes	<code>eas build --platform ios</code> compiles in Expo's cloud on macOS workers
Manage certificates	Yes	EAS handles provisioning automatically
Submit to App Store	Yes	<code>eas submit --platform ios</code>
Test the build	No	TestFlight requires an iOS device. Borrow one, buy a cheap secondhand model, or recruit an iPhone-owning tester

Prerequisites: an Apple Developer Program membership (~R1,800/year) and access to at least one iPhone.

Additional iOS-specific work when you get there:

- App Store Review Guideline **1.4.1** is the real risk: Apple rejects apps that "could be used for diagnosing or treating patients" without validated accuracy or regulatory clearance. Your Beauty categorisation and non-medical vocabulary are the defence. Expect at least one rejection and a reframing round; budget for it.
- Apple requires **Sign in with Apple** if you offer any other third-party sign-in. Magic-link email only does not trigger this.
- A privacy manifest file declaring API usage and data collection.
- App Tracking Transparency, only if you add attribution SDKs. Skip them and skip this.
- iOS pricing tiers differ slightly; reconfigure in App Store Connect and RevenueCat.

P15.1 — iOS build preparation

PASTE INTO CLAUDE CODE

Prepare the project for an iOS build from Windows via EAS.

1. Add the `ios` section to `app.json`: `bundleIdentifier`, `buildNumber`, and usage description strings for camera and photo library that explain the purpose in plain language a reviewer will accept.
2. Add an `"ios"` production profile to `eas.json`.
3. Audit every dependency for iOS compatibility and report anything that is Android-only or needs configuration.
4. Create the privacy manifest file (`PrivacyInfo.xcprivacy`) declaring the required-reason APIs we use and the data types we collect.
5. Review every screen for iOS-specific layout issues: safe areas including the dynamic island, the home indicator, and the absence of a hardware back button – flag any screen that relies on Android back behaviour.
6. List everything that needs doing in App Store Connect before submission.

Do not change any shared code unless it is genuinely broken on iOS.

Commit: `build: prepare ios build configuration`

21. Launch and marketing

Distribution is the moat. The app is cloneable in a weekend; creator relationships and an accumulating progress history are not.

The content format that works

Every breakout app in this category grew on the same 15-second structure:

0-2s	HOOK	face to camera, a specific complaint "I've spent thousands on skincare and I still have no idea if any of it is working"
2-6s	DEMO	screen recording: open app, point camera, capture
6-11s	REVEAL	the score animating in -- this is the whole video. Genuine reaction. Do not fake it.
11-15s	PAYOFF	the before/after slider or the trend graph "this is six weeks apart"
	CTA	name the app once, clearly, in the caption

The reveal animation is your advertisement. This is why Phase 7 deserved disproportionate effort. If the score ring landing does not feel satisfying on camera, no amount of marketing spend fixes it. Watch your own launch video with the sound off — if the middle five seconds are boring, go back and fix the animation.

Launch sequence

Timing	Action
T-21 days	Create accounts on TikTok, Instagram Reels and YouTube Shorts. Start posting build-in-public content now, so the account is not brand new on launch day
T-14 days	Identify 30-50 micro-creators (5k-50k followers) in skincare. Smaller accounts convert better per rand than large ones and will actually reply
T-10 days	Reach out. Offer free lifetime access plus a modest flat fee. Expect roughly one reply in five
T-7 days	Film 10-15 of your own videos. Do not post them yet. Having a backlog is what lets you post daily through launch week
T-3 days	Confirm creators. Send them a brief, the app, and what to show. Do not script them word for word — it reads as an advert and performs badly
Day 0	Your posts go live. Creator posts go live the same day. Post to relevant subreddits where self-promotion is allowed
Day 1-7	Post at least twice daily. Reply to every single comment — early engagement drives distribution more than anything else
Day 7-30	Double down on whatever format worked. Kill what did not. Start the second creator wave with the winning brief

Budget guidance

At the small end, roughly R9,000-R18,000 buys a meaningful first wave: 10-15 micro-creators at R450-R900 each, plus a little paid amplification on your best-performing organic video. Precedent in this category suggests unknown creators at small flat fees outperform expensive ones.

Do not spend on paid acquisition until the funnel is proven. Until you can see, in PostHog, that install-to-trial and trial-to-paid are healthy, every rand of ad spend is buying you data you could have got for free from organic. Get the funnel right first, then pour fuel on it.

The organic engine you should build

Once launched, your cheapest growth channel is your own users:

- **The shareable progress card** (built in P7.2). Every share is a free, credible advert from a real person.
- **A referral loop**: one free week for both sides. Cheap, and it converts because the referrer is recommending something they are already paying for.
- **Milestone prompts**: after a visible improvement, prompt a share. Timing matters — ask at the moment of the win.

22. Metrics and kill criteria

The numbers to watch

Metric	Healthy	Worrying	Measured by
Install to onboarding complete	>70%	<50%	PostHog activation funnel
Onboarding to first scan	>60%	<40%	PostHog activation funnel
Camera open to scan submitted	>70%	<50%	Capture funnel
Scan rejection rate	<15%	>30%	scan_rejected events
Paywall view to trial start	>8%	<4%	Monetisation funnel
Trial to paid	>25%	<10%	RevenueCat
Week 1 re-scan rate	>40%	<20%	Retention loop funnel
Week 4 re-scan rate	>25%	<10%	Retention loop funnel
Day 30 paid retention	>60%	<35%	RevenueCat cohorts
Month 12 annual retention	>25%	<15%	RevenueCat cohorts
Crash-free sessions	>99.5%	<99%	Sentry
Scan latency (p90)	<12s	>25s	scan_completed events

Kill criteria

Set these now, while you are unattached to the outcome. Decide in advance what evidence would change your mind, and write it down — otherwise you will rationalise indefinitely.

After a genuine launch with creator seeding and at least 1,000 installs:

- **Trial-to-paid below 5%** — the value proposition is not landing. Two pivots to try before killing: reprice, or move the paywall later (after the second scan rather than the first).
- **Week-4 re-scan below 10%** — the retention loop has failed. This is the fatal one, because it means you built a novelty, not a subscription. Fixing it means changing the core product, not the marketing.
- **Repeated store rejection after non-medical reframing** — if Google or Apple will not accept the category positioning after two genuine attempts, the regulatory surface is larger than a solo developer can carry.
- **Scan-to-scan score variance above 8 points on a controlled repeat** — the measurement is noise. If you cannot fix this at the model layer, the product's central promise is false and should not ship.

Distinguish a marketing failure from a product failure. Low installs with good retention is a marketing problem and entirely solvable. High installs with collapsing retention is a product problem and is not solved by more installs. The second is the one that ends projects, so watch the retention numbers harder than the download numbers.

Weekly review ritual

Thirty minutes, same time every week:

1. Pull the four funnels from PostHog. Note the biggest week-on-week change.
2. Check the RevenueCat cohort chart. Is the newest cohort retaining better or worse than the last?

3. Read every new store review. All of them.
4. Check Sentry for any new crash affecting more than 1% of sessions.
5. Write down the single biggest bottleneck, and make it next week's priority.

23. Risk register

#	Risk	Likelihood	Impact	Mitigation
1	Store rejection on medical grounds	Medium	High	Beauty category, banned-vocabulary list in CLAUDE.md, disclaimers everywhere, compliance audit before every submission
2	12-tester gate delays launch	High	Medium	Start the closed test in week 3 on an unfinished build. Over-recruit to 15. Hold a paid service in reserve
3	Model scoring inconsistency	Medium	Critical	Calibration harness (P6.3) is a hard gate. Low temperature, anchored rubric, server-side recomputation of overall
4	Poor retention after novelty	High	Critical	The entire product is designed around the re-scan loop: weekly reminders, streaks, trend graph, daily routine
5	API key extracted from the app	Low	High	Key exists only in Edge Function secrets. Server-side entitlement check caps abuse
6	Face photograph data breach	Low	Critical	Private bucket, RLS verified manually, signed URLs only, real deletion, minimal retention
7	Keystore lost	Low	Critical	Back up in two places, one offline, immediately after the first build
8	Copcats	High	Medium	The moat is distribution and accumulated user history, not code. Ship fast, build creator relationships
9	Paywall flagged as predatory	Low	High	Honest pricing display, visible close button, no fake urgency, plain renewal terms
10	Gemini pricing or model changes	Medium	Low	Model abstraction in one file. Re-run calibration after any model change
11	Seasonality (January and spring spikes)	Certain	Low	Plan for uneven months. Time the launch and major pushes to January or early spring
12	Solo-founder burnout	Medium	High	The phase structure exists so you can stop and resume cleanly. Ship the MVP; do not gold-plate
13	Supabase free-tier project pauses	Low	Low	Harmless in development. Move to Pro before the closed test begins
14	POPIA/GDPR data-subject request mishandled	Low	Medium	In-app export and deletion built in Phase 12. Document the manual process too

Appendix A — CLAUDE.md reference

The compliance section is the part that matters most. Paste this into your **CLAUDE.md** and expand around it.

```
## COMPLIANCE -- READ THIS BEFORE WRITING ANY USER-FACING TEXT
```

This is a BEAUTY app. It is not a medical app, not a health app, and not a diagnostic tool. Store approval and the legal viability of the product both depend on this distinction holding in every single string we ship.

NEVER use these words in any user-facing string, code comment, variable name, commit message, store listing, or model prompt:

diagnose	diagnosis	condition	disease
disorder	symptom	treat	treatment
cure	heal	therapy	medical
clinical	patient	prescription	lesion
dermatologist-grade		medically proven	
acne vulgaris	rosacea	eczema	psoriasis
dermatitis	melasma		

ALWAYS use this vocabulary instead:

"the appearance of"	"looks"	"visible"
"cosmetic"	"routine"	"care"
"clarity"	"texture"	"evenness"
"glow"	"skincare"	

NEVER make an accuracy claim. No percentages. No "proven", "precise", "scientifically", "clinically". No comparison to a professional assessment.

EVERY screen that displays a score must render the Disclaimer component, which reads from src/constants/copy.ts. No exceptions, no collapsing it, no hiding it behind a link.

ALL user-facing strings live in src/constants/copy.ts. Never hardcode one in a component. This single file is the compliance chokepoint -- reviewing it proves the whole app is compliant.

Appendix B — Git workflow

```
# Start a phase
git checkout main
git pull
git checkout -b phase-08-progress

# After each prompt that passes its Done-when
git add -A
git commit -m "feat(progress): trend chart and scan history"

# If a prompt makes a mess
git reset --hard HEAD      # discard uncommitted changes
git reset --hard HEAD~1    # undo the last commit entirely

# Finish a phase
npx tsc --noEmit           # must pass
git push -u origin phase-08-progress
# open a PR on GitHub, check CI is green, merge

# Tag a release build
git tag -a v1.0.0 -m "First production build"
git push --tags
```

Commit message prefixes: **feat**: new functionality, **fix**: a bug, **chore**: tooling or config, **docs**: documentation, **refactor**: no behaviour change, **build**: build configuration, **ci**: pipeline.

Appendix C — Troubleshooting

Symptom	Likely cause and fix
Expo Go cannot connect	Different Wi-Fi networks, or client isolation on the network. Use <code>npx expo start --tunnel</code>
Metro bundler errors after installing a package	<code>npx expo start --clear</code> . If it persists, delete <code>node_modules</code> and <code>package-lock.json</code> , then <code>npm install</code>
Project in OneDrive behaves strangely	Move it to <code>C:\dev\</code> . OneDrive sync corrupts <code>node_modules</code>
"Unable to resolve module @/..."	Path alias mismatch between <code>tsconfig.json</code> and the Babel/Metro config. Both must agree
Supabase query returns empty but rows exist	RLS policy is blocking. Check <code>auth.uid()</code> matches the row's <code>user_id</code>
Edge Function returns 500 with no detail	<code>supabase functions logs analyze-scan</code> . Usually a missing secret
Purchases do not appear in a debug build	Expected. Play billing only works in a build uploaded to a Play track and signed with the release key
Play products show as unavailable	Propagation delay (up to several hours), or the account is not licence-tested, or the build is not on a track yet
EAS build fails on credentials	<code>eas credentials</code> to inspect and regenerate
Model returns valid JSON that fails the schema	Tighten <code>responseSchema</code> , lower the temperature, and check the retry path is firing
Scores swing wildly between identical photos	Temperature too high, or the scoring rubric is too vague. Re-run the calibration harness

Appendix D — Prompt index

#	Prompt	Phase
P1.2	Scaffold the project	1
P1.3	Write CLAUDE.md	1
P1.4	Continuous integration	1
P2.1	Theme tokens	2
P2.2	UI primitives	2
P2.3	The copy file	2
P3.2	Schema and row-level security	3
P3.3	Private storage bucket	3
P3.4	Typed Supabase client	3
P4.1	Anonymous-first authentication	4
P4.2	Onboarding flow	4
P5.1	Camera screen	5
P5.2	Ghost overlay	5

#	Prompt	Phase
P6.1	The Edge Function	6
P6.2	The vision system prompt	6
P6.3	Calibration harness	6
P6.4	Wire the client to the pipeline	6
P7.1	Results screen	7
P7.2	Shareable progress card	7
P8.1	Progress screen	8
P8.2	Before/after comparison	8
P9.1	Routine generation and display	9
P9.2	Notifications	9
P10.2	Paywall implementation	10
P10.3	Server-side entitlement check	10
P11.1	Event instrumentation	11
P12.1	In-app compliance surfaces	12
P12.2	Privacy policy and terms	12
P12.3	Compliance audit	12
P13.1	Hardening pass	13
P13.2	Manual test script	13
P14.1	First EAS build	14
P14.3	Store listing copy	14
P15.1	iOS build preparation	15

Appendix E — What to do first

If you read nothing else, do these five things this week.

1. **Register the Google Play developer account today.** Identity verification is the longest-lead item in the entire plan and everything else waits behind it.
2. **Decide the app name and bundle ID.** The bundle ID is permanent. Check availability on both stores and as a domain.
3. **Work through Phase 0.** Get an Expo app running on your phone. It takes an evening and it de-risks everything after it.
4. **Start a tester list.** Open a note and write down 20 people who might test it. You need 12 to actually follow through, and you will start asking in week 3.
5. **Run Phases 1 through 3.** By the end of week one you should have a scaffolded app, a design system and a live database with row-level security.

Then keep going in order. The phases are sequenced so that each one is testable on its own, and so that you can stop at the end of any phase and resume weeks later without having lost the thread.